PSI 2025-2026 – Objectif 2 : Little

Résumé

Ce document donne quelques instructions et conseils pour vous guider dans la réalisation de l'objectif 2, qui consiste à implémenter l'algorithme de Little en C#.

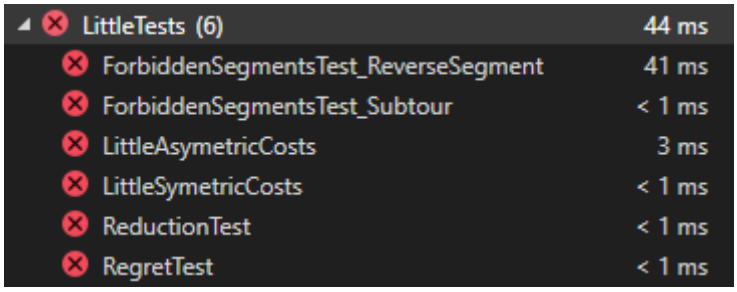
1 – Eléments fournis

Pour vous aider dans cet objectif, les éléments suivants vous sont fournis :

- Classe `Little` : implémentation de l'algorithme de Little (patron à compléter)
- Classe `Tour` : modélisation d'une tournée (patron à compléter) ; résultat de l'algorithme de Little
- Classe `LittleTests` : tests unitaires pour valider le bon fonctionnement de l'algorithme de Little :
    - Etapes clés
    - Résolution d'un problème symétrique (celui du cours) et asymétrique (celui de ce TD).

Pour intégrer ces éléments à votre projet, **un** des membres du groupe doit réaliser les opérations suivantes :

1. Récupérer le fichier ZIP contenant ces éléments sur DVL, à cette page.
2. Dézipper le fichier
3. Ouvrir votre projet PSI dans Visual Studio
4. Glisser-déposer un a un les fichiers Little.cs et Tour.cs dans le projet « TourneeFutee » (dans le panneau « explorateur de solutions »)
5. Glisser-déposer le fichier LittleTests.cs dans le projet « TourneeFutee.Tests »
6. Lancer les tests de la campagne « LittleTests » pour vérifier que tout fonctionne : tout doit compiler, les tests doivent se lancer et échouer, comme illustré ci-dessous :



Puis, pour que les autres membres du groupe puissent en profiter :

1. Valider les modifications de type ajout dans votre dépôt local (*commit*)
2. Pousser les modifications sur le dépôt distant (*push*)
3. Demander aux autres membres du groupe de tirer les modifications (*pull*).

A présent, tous les membres ont un projet à jour en local.

2 – Exercice d'appropriation

Avant de tenter d'implémenter l'algorithme de Little, nous vous conseillons de dérouler l'exemple suivant à la main pour bien comprendre et vous approprier chacune des étapes.

Dans cet exemple, les coûts sont asymétriques, et l'arbre de recherche comprend le développement d'un nœud de type « trajet exclu ».

Cet exemple fait partie des deux exemples qui seront utilisés dans les tests unitaires pour évaluer votre travail.

Définition du problème

- Villes : A, B, C, D, E, F
- Matrice des coûts :

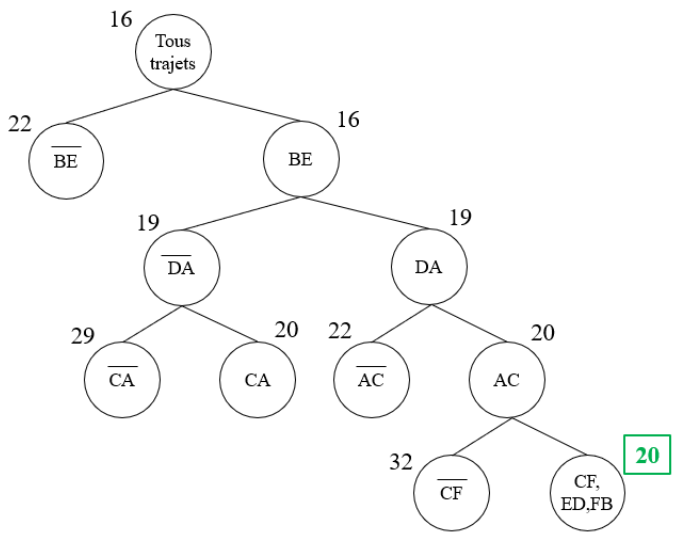
	A	B	C	D	E	F
A	+∞	1	7	3	14	2
B	3	+∞	6	9	1	24
C	6	14	+∞	3	7	3
D	2	3	5	+∞	9	11
E	15	7	11	2	+∞	4
F	20	5	13	4	18	+∞

Solution partielle

- Tournée optimale (au départ de A) :

AC-CF-FB-BE-ED-DA (coût : 20)

- Arbre de recherche :



3 – Implémentation des étapes clés

Dans la classe `Little`, commencez par implémenter les méthodes suivantes, qui correspondent à quelques étapes clés de l'algorithme :

Méthode	Rôle
`ReduceMatrix`	Réduit la matrice de coûts et revoie la valeur totale de la réduction
`GetMaxRegret`	Retourne la localisation et la valeur du regret maximal dans une matrice de coûts réduite
`IsForbiddenSegment`	Indique si un trajet est « parasite », c'est-à-dire s'il ferme prématurément une tournée en cours de construction.

Au fur et à mesure des implémentations, vous pouvez progressivement tester ces méthodes avec les tests unitaires correspondants dans la campagne `LittleTests` :

Méthode	Test unitaire dans LittleTests
`ReduceMatrix`	`ReductionTest`
`GetMaxRegret`	`RegretTest`
`IsForbiddenSegment`	`ForbiddenSegmentsTest_ReverseSegment` `ForbiddenSegmentsTest_Subtour`

4 – Implémentation de l'algorithme

Conception

Avant de coder l'algorithme, imaginez-en la structure.

Une manière naturelle de mettre en œuvre l'algorithme de Little est d'utiliser la récursivité : faire un appel récursif pour le trajet exclu, et un pour le trajet inclus.

Si vous optez pour cette solution, réfléchissez aux paramètres et aux conditions d'arrêt de votre fonction récursive.

Implémentation

Dans la classe `Little`, implémenter le corps de la méthode :

```
public Tour ComputeOptimalTour()
```

qui calcule la tournée optimale dans le graphe renseigné au moment d'instancier le planificateur, en utilisant l'algorithme de Little.

Cette méthode doit retourner une instance de la classe `Tour` fournie.

Conseils :

- En C#, des constantes existent pour les infinis, par exemple `float.PositiveInfinity` pour le type `float`
- Chaque appel récursif travaille sur une **copie** de la matrice de coûts, qui est ensuite modifiée selon le type de trajet (inclus, exclu)
- N'hésitez pas à ajouter toutes les méthodes qui vous paraissent utiles aux types `Matrix` et `Graph` (du moment que cela ne casse pas les tests unitaires)
- N'hésitez pas à ajouter vos propres classes, pour modéliser différents concepts manipulés par l'algorithme de Little

Test

Lancez les tests `LittleSymetricCosts` et `LittleAsymetricCosts` de la campagne `LittleTests` pour vérifier le bon fonctionnement de votre implémentation.

5 – Mise à jour du diagramme de classes

Mettez à jour le diagramme de classes UML démarré dans l'objectif 1, en ajoutant les classes fournies, les classes que vous avez ajoutées, et leurs liens avec les classes existantes.

Cette partie n'est pas évaluée tout de suite, mais l'anticiper dès à présent permettra de diminuer votre charge de travail lors de la rédaction du rapport final.

6 – Rendu

Soumissions régulières : sur GitHub

Soumettez régulièrement votre travail sur le dépôt Git.

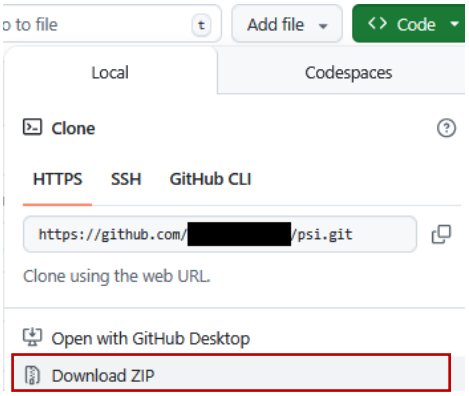
Tous les membres doivent contribuer au dépôt. Les commits de chaque membre permettront notamment à votre encadrant d'apprécier votre apport au projet.

Dépôt final pour évaluation : zip sur DVL

Le travail évalué sera votre dépôt sur DVL.

Pour rendre votre travail sur DVL :

1. Allez sur la page GitHub de votre dépôt
2. Cliquez sur « Code » puis « Download zip »



3. Renommez votre fichier zip en psi_objectif2_groupeX.zip, où X est votre nom de groupe (attribué au début du module). Par exemple, psi_objectif2_groupeNA1.zip pour Nantes, groupe de TD A, groupe de projet 1.
4. Déposez votre fichier zip sur DVL, activité « Rendu objectif 2 », en utilisant le compte du **représentant du groupe** (ne faire qu'un dépôt, à ce nom).

Date limite de rendu : lundi 6 avril 2026, 23:59 Pénalités de retard : 1 point par heure de dépassement de la deadline.

Conseil : n'attendez pas le dernier moment pour soumettre votre travail : vous pouvez faire autant de rendus que vous le souhaitez dans la plage de dates autorisée.

7 – Evaluation

L'objectif 2 est uniquement évalué sur la réussite aux tests unitaires.

- Campagne de tests : `LittleTests`

- Barème :

Test unitaire	Elément testé	Points
ReductionTest	Réduction de la matrice de coûts	2
RegretTest	Calcul du regret maximal	2
ForbiddenSegmentsTest_ReverseSegment	Détection d'un trajet parasite dans un cas simple (trajet inverse)	2
ForbiddenSegmentsTest_Subtour	Détection d'un trajet parasite dans un cas complexe (sous-tournée)	4
LittleSymetricCosts	Calcul de tournée par l'algorithme de Little sur un problème symétrique (cours)	5
LittleAsymetricCosts	Calcul de tournée par l'algorithme de Little sur un problème asymétrique (TD)	5

- Si le code ne compile pas ⇒ Note = 0

Vous connaissez donc votre note avant de rendre votre travail.